

Практическая работа № 2

Разработка многостраничного веб-приложения на PHP с авторизацией, CRUD-операциями

Цель работы

Закрепить практические навыки разработки серверной части веб-приложений на языке PHP: реализовать маршрутизацию, авторизацию пользователей с хешированием паролей, CRUD-операции над данными в JSON-хранилище, работу с сессиями и cookies, разделение логики и представления через шаблонизацию, а также обеспечить базовый уровень безопасности приложения.

Задачи работы

1. Настроить локальное окружение для запуска PHP-приложения.
2. Спроектировать и реализовать архитектуру многостраничного приложения с единой точкой входа и маршрутизацией.
3. Реализовать систему регистрации и авторизации пользователей с использованием `password_hash` и `password_verify`.
4. Организовать хранение данных пользователей и задач в отдельных JSON-файлах, реализовать CRUD-операции.
5. Разделить логику и представление с помощью PHP-шаблонов (header/footer).
6. Обеспечить контроль доступа к страницам в зависимости от статуса авторизации пользователя.
7. Реализовать счётчик посещений профиля через cookies.
8. Оформить интерфейс приложения с помощью CSS-стилей.
9. Провести тестирование всех функциональных модулей и подтвердить корректность работы.

Используемые инструменты и технологии

- Язык программирования: PHP 8.x.
- Веб-сервер: встроенный сервер PHP (`php -S`) либо Apache (OpenServer/XAMPP/MAMP).
- Текстовый редактор: VS Code, Sublime Text, Atom.
- Формат хранения данных: JSON.
- Механизмы состояния: сессии PHP, cookies.
- Методы HTTP: GET, POST.
- Функции безопасности: `htmlspecialchars()`, `password_hash()`, `password_verify()`.

Ход работы

Этап 1. Подготовка окружения и структуры проекта

Создайте папку проекта `lab_php_advanced` и внутри неё следующую структуру:

Папка `storage` и файлы `JSON` будут созданы автоматически при первом обращении к функциям хранения.

Запуск

сервера:

В терминале перейдите в папку проекта и выполните:

Откройте в браузере `http://localhost:8000/`.

Этап 2. Реализация конфигурации и маршрутизации

В файле `config.php` настройте пути к хранилищам, параметры сессии и включите вывод ошибок для разработки. В `index.php` реализуйте маршрутизацию на основе параметра `?page=название_страницы`, проверку прав доступа и подключение соответствующей страницы. Для Apache создайте файл `.htaccess` с правилами перезаписи URL.

Результат этапа: приложение корректно обрабатывает запросы к разным страницам, возвращает 404 при несуществующем маршруте, перенаправляет неавторизованных пользователей на страницу входа.

Этап 3. Шаблонизация и главная страница

Создайте файлы `templates/header.php` и `templates/footer.php` для единообразного оформления всех страниц. В них реализуйте динамическую навигацию: отображение ссылок «Регистрация/Вход» для гостей и «Задачи, Профиль, Выйти» для авторизованных пользователей. На странице `pages/home.php` выведите приветственное сообщение и подсказки по дальнейшим действиям.

Результат этапа: все страницы используют общие шаблоны, навигация адаптируется под статус пользователя, HTML-код не дублируется.

Этап 4. Реализация библиотеки работы с хранилищем

В файле `lib/storage.php` создайте функции для чтения, записи, создания, поиска, обновления и удаления записей в JSON-файле. Используйте `json_encode()` с флагами `JSON_UNESCAPED_UNICODE` и `JSON_PRETTY_PRINT`, а также блокировку файла при записи (`LOCK_EX`).

Результат этапа: JSON-файлы корректно хранят массивы данных, каждая запись имеет уникальный ID, операции CRUD работают без потери или повреждения данных.

Этап 5. Валидация пользовательских данных

В файле `lib/validation.php` реализуйте функции проверки email, имени, пароля и названия задачи. Каждая функция должна возвращать сообщение об ошибке или `null` при успешной проверке. Создайте вспомогательную функцию `collectErrors()` для сборки списка ошибок из массива проверок.

Результат этапа: формы не принимают некорректные данные, пользователь получает понятные сообщения об ошибках, введённые значения сохраняются в полях формы при ошибке.

Этап 6. Авторизация и управление доступом

В файле `lib/auth.php` реализуйте функции `isLoggedIn()`, `currentUser()`, `registerUser()`, `loginUser()` и `logoutUser()`. Используйте `password_hash()` для сохранения паролей и `password_verify()` для проверки. В `index.php` добавьте проверку прав доступа к маршрутам: защищённые страницы доступны только авторизованным пользователям, страницы регистрации и входа — только гостям.

Результат этапа: пользователи могут регистрироваться и входить в систему, пароли хранятся в хешированном виде, доступ к страницам контролируется, при попытке доступа без прав происходит редирект.

Этап 7. CRUD задач с изоляцией данных

На странице `pages/tasks.php` реализуйте создание, просмотр, редактирование и удаление задач. Каждая задача должна содержать поля: `user_id`, `title`, `description`, `status`, `created_at`. При выполнении операций проверяйте, что пользователь является владельцем задачи. Отображайте задачи в виде таблицы с возможностью редактирования и удаления.

Результат этапа: пользователи видят только свои задачи, операции CRUD выполняются корректно, данные не пересекаются между пользователями.

Этап 8. Профиль пользователя и счётчик посещений

На странице `pages/profile.php` выведите информацию о текущем пользователе: имя, email, дату регистрации, общее количество задач и количество выполненных. Реализуйте счётчик посещений профиля с помощью cookies: при каждом открытии страницы увеличивайте значение cookie и отображайте его на экране.

Результат этапа: профиль отображает актуальные данные пользователя, счётчик посещений корректно увеличивается и сохраняется между сеансами, у каждого пользователя свой счётчик.

Этап 9. Оформление интерфейса

В файле `style.css` добавьте стили для всех элементов: шрифты, отступы, цвета, таблицы, формы, кнопки, сообщения об ошибках. Используйте семантические классы для статусов задач (новая, в работе, выполнена) и оформите их разными цветами.

Результат этапа: интерфейс приложения выглядит аккуратно и единообразно, все элементы удобно расположены, сообщения об ошибках и статусы задач визуально различимы.

Требования к отчёту

Отчёт по практической работе должен содержать:

1. Титульный лист с указанием темы, дисциплины, специальности, курса, группы, Ф. И. О. студента и преподавателя.
2. Цель и задачи работы.
3. Описание структуры проекта и используемых технологий.
4. Листинги ключевых фрагментов кода (не более 3–4 строк для каждой функции, остальное — в приложении).
5. Скриншоты интерфейса приложения: главная страница, регистрация, вход, список задач, профиль пользователя.
6. Описание результатов тестирования: проверка всех сценариев (успешные и ошибочные), подтверждение изоляции данных между пользователями, проверка работы сессий и cookies.
7. Вывод о достижении цели работы и приобретённых навыках.
8. Архив с полным кодом проекта (все файлы и папки).

Критерии оценки

Критерий	Баллы
Архитектура и структура проекта	15
Реализация авторизации и безопасности	20
CRUD-операции и изоляция данных	20
Валидация и обработка ошибок	15
Шаблонизация и оформление интерфейса	15
Тестирование и отчёт	15
Итого	100

Контрольные вопросы для защиты работы

1. Как работает механизм сессий в PHP? Чем отличаются сессии от cookies?
2. Почему важно использовать `password_hash()` вместо простого сохранения паролей в открытом виде?
3. Что такое XSS-атаки и как функция `htmlspecialchars()` помогает защититься от них?
4. Объясните принцип работы CRUD-операций и приведите примеры их реализации в вашем проекте.
5. Как вы обеспечили изоляцию данных между пользователями?
6. Опишите процесс маршрутизации в вашем приложении и объясните, зачем нужна единая точка входа.
7. Какие методы валидации данных вы использовали и почему?
8. Как работает счётчик посещений через cookies? Какие риски связаны с использованием cookies?
9. Какие дополнительные меры безопасности можно внедрить в подобное приложение?
10. Как бы вы масштабировали это приложение для работы с большим количеством пользователей?

Дополнительные задания !!!!!

- Реализовать поиск и фильтрацию задач по статусу и названию.
- Добавить пагинацию для списка задач (по 5 записей на страницу).
- Внедрить загрузку аватара пользователя с проверкой типа и размера файла.
- Создать REST API endpoint для получения задач пользователя в формате JSON.

- Вести лог действий пользователей (создание, удаление задач) в отдельном файле.
- Ограничить удаление задач, созданных более 24 часов назад.

Вывод

В ходе выполнения практической работы было разработано многостраничное веб-приложение на PHP, реализующее полный цикл CRUD-операций, систему авторизации с хешированием паролей, управление состоянием через сессии и cookies, а также разделение логики и представления. Приложение соответствует базовым требованиям безопасности и предоставляет удобный интерфейс для управления задачами. Полученные навыки могут быть использованы при разработке более сложных серверных приложений.